

EXPERIMENT 5

Case Study on Analysis of Data Communication in a Web Application Using Browser Developer Tools

Aim

To analyze how data is communicated between a web browser and a web server using Browser Developer Tools, focusing on HTTP/HTTPS requests, responses, headers, payloads, status codes, and network performance.

Objectives

- Understand data communication in web applications.
- Analyze HTTP/HTTPS requests and responses.
- Examine GET, POST, PUT, and DELETE requests.
- Study headers, status codes, and payloads.
- Understand API-based client-server communication.
- Analyze network performance and identify communication errors.

Problem Statement

Modern web applications continuously exchange data between browsers and servers. Users and beginners often cannot see what is transmitted, which server receives it, what response is returned, or why a request fails. The Network panel of Browser Developer Tools provides a practical way to observe this communication and troubleshoot problems.

Selected Web Application

An online food delivery web application is used as an example because it involves browser-server communication for login, restaurant search, cart operations, orders, payments, and API calls.

Communication Flow

User → Browser → HTTP/HTTPS Request → Web Server/API → Database or External Service → HTTP/HTTPS Response → Browser → Information displayed to User

Browser Developer Tools

Tool	Purpose
Elements	Examines HTML and CSS
Console	Displays JavaScript messages and errors
Network	Analyzes data communication
Sources	Examines JavaScript and source files
Application	Examines storage, cookies, and application data
Performance	Analyzes webpage performance

Network Panel Analysis

The Network panel displays request URL, method, status code, headers, payload, response data, initiator, loading time, and transferred size.

Request	Method	Status	Purpose
/restaurants	GET	200	Retrieve restaurants
/login	POST	200	Authenticate user
/cart	GET	200	Retrieve cart
/order	POST	201	Create order
/payment	POST	200	Process payment

HTTP Request Methods

Method	Purpose	Example
GET	Retrieve data	Get restaurant list
POST	Create/send data	Place an order
PUT	Update data	Update user profile
DELETE	Remove data	Remove cart item

HTTP Status Code Analysis

Code	Meaning	Example
200	OK	Data retrieved
201	Created	New order created
400	Bad Request	Invalid request data
401	Unauthorized	Authentication required
403	Forbidden	Access denied
404	Not Found	Resource unavailable
500	Internal Server Error	Server-side problem

Request and Response Example

Request: GET /api/restaurants?location=Delhi

Response: 200 OK

Example JSON response: { restaurants: [{ name: "Restaurant A", rating: 4.5 }, { name: "Restaurant B", rating: 4.2 }] }

HTTPS Communication

HTTPS encrypts communication between browser and server and is important for credentials, personal information, authentication tokens, and payment-related information.

Performance Analysis

Request	Loading Time	Status	Observation
Homepage	420 ms	200	Normal
Restaurant API	650 ms	200	Moderate
Image Request	1.2 s	200	Large resource
Order API	380 ms	201	Fast
Payment API	720 ms	200	Moderate

Benefits

- Real-time visibility into network communication.
- Helps identify failed API requests.
- Helps debug frontend-backend communication.
- Identifies slow requests and supports performance optimization.
- Assists in API response analysis.

Limitations

- Primarily provides client-side visibility.
- Server-side diagnosis may require backend logs.
- Large applications can generate many requests.
- Understanding API communication requires basic HTTP, JSON, and web architecture knowledge.

Conclusion

Browser Developer Tools provide an effective way to analyze data communication in web applications. The Network panel exposes useful information about methods, URLs, headers, payloads, responses, status codes, and loading times. This makes the tools valuable for debugging, API testing, performance analysis, and understanding frontend-backend communication.